



Programmieren 2 - 3. Test - Vorbereitung A Sommersemester 2026

Allgemeine Hinweise:

Elektronische Geräte sind abzuschalten und schwer erreichbar (z.B. in einer verschlossenen Tasche, aber **nicht** am Körper) zu verstauen (zur Not am Vortragendenpult hinterlegen).

Die Prüfungsaufgaben befinden sich auf der Rückseite der Angabe.

Vorbereitung:

Die folgenden Aufgaben sind in eclipse zu lösen. Kopieren Sie die beiden Dateien Main.java und Checks_Do_Not_Change.java in Ihre Implementierung des Java Projekts (Ein eventuell vorhandenes Main ist zuvor umzubenennen oder zu löschen.) Ändern Sie den Namen des Packages (**package** Anweisung) in Main und in Checks_Do_Not_Change auf Ihren package Namen (a+Matrikelnummer).

Erlaubte Hilfsmittel:

Zugriff auf die Webseite der Lehrveranstaltung und auf die Java Dokumentation (<https://docs.oracle.com/en/java/javase/17/docs/api>) ist erlaubt. Andere Webseiten dürfen nicht zugegriffen werden. Es dürfen neben dem Webbrowser und PDF-Viewer nur Programme verwendet werden, die für die Lösung der Aufgabenstellung notwendig sind (Eclipse und Terminal). Als Java-Referenz kann auch das in Eclipse integrierte Hilfe-System verwendet werden.

Die Bearbeitungszeit beträgt 90 Minuten. Alle Instanzvariablen sind **private** zu deklarieren. Veränderungen an den von der LV-Leitung vorgegebenen Klassen sind (soweit nicht explizit in der Aufgabenstellung verlangt) nicht erlaubt. Es werden ausschließlich allgemeingültige Lösungen für Probleme akzeptiert. Die Verwendung von `instanceof`, `getClass`, `isAssignable` etc. zur Lösung der Aufgabenstellungen führt zu Punktabzug. Die Abgabe muss kompilieren. Es werden nur die Aufgaben bewertet die in der `main` Methode der Main-Klasse dekommentiert wurden.

Sie dürfen bei allen Aufgaben davon ausgehen, dass alle übergebenen Parameter sinnvolle Werte sind und keine null-Objekte auftreten.

1. (1 Punkt) Kompletieren Sie die in der Klasse `Main` vordefinierte Klassenmethode `Potion[] sortPotions(List<Potion> potions)` so, dass die Liste `potions` in drei Teile A, B und C aufgeteilt wird. A enthält alle Objekte aus `potions`, deren Preis (`getPrice`) durch drei teilbar ist, aufsteigend nach Gewicht (`getWeight`) sortiert. B enthält alle Objekte aus `potions`, deren Preis bei ganzzahliger Division durch drei Rest 1 ergibt, in derselben relativen Reihenfolge wie in `potions`. C enthält alle Objekte aus `potions`, deren Preis bei ganzzahliger Division durch drei Rest 2 ergibt, absteigend nach Gewicht sortiert. Die drei Teillisten werden dann zum Füllen eines Arrays verwendet, wobei aus jeder Teilliste in der Reihenfolge A, B und C jeweils das nächste Element in das Array übertragen wird. Teillisten, aus denen schon alle Objekte übertragen wurden, werden einfach ignoriert. Das Verfahren wird so lange fortgesetzt, bis alle Teillisten vollständig abgearbeitet wurden. Das so gefüllte Array, dessen Größe exakt der Anzahl der enthaltenen Objekte entsprechen muss, ist zu retournieren. Es bleibt Ihnen überlassen, welche Vorgangsweise für das Sortieren der Werte Sie wählen.
2. (1 Punkt) Kompletieren Sie die in der Klasse `Main` vordefinierte Klassenmethode `TreeMap<Integer, TreeSet<MagicItem>> mapItemsPerPriceCategory(List<MagicItem> items)` so, dass eine `TreeMap` retourniert wird, die jeder in `items` auftretenden Preiskategorie als Schlüssel, ein `TreeSet` aller `MagicItem`-Objekte aus `items` in der entsprechenden Kategorie als Wert zuordnet. Die Preiskategorie ergibt sich, als erste Ziffer des Preises (z. B. Preis 29, 2 oder 247 ergibt Preiskategorie 2, Preis 7, 71 oder 777 ergibt Preiskategorie 7 usw.). Die Sortierreihenfolge im `TreeSet` ist so zu definieren, dass die Objekte nach Gewicht absteigend sortiert werden. Sie können die natürliche Reihenfolge von `MagicItem` Objekten entsprechend definieren (Implementieren des Interfaces `Comparable`) oder ein geeignetes `Comparator`-Objekt passend verwenden.
3. (1 Punkt) Erstellen Sie eine Klasse `AttackingProtectionSpell`, die von `AttackingSpell` erbt. Die Methode `cast` ist so zu override, dass prinzipiell dasselbe passiert wie in der Basisklasse, danach aber zusätzlich, unabhängig davon, ob der Zauber erfolgreich ausgeführt werden konnte oder nicht, ein Schutz gegen diesen Zauber (`this`-Objekt) auf dem Objekt, das den Zauber wirkt (`source`), aufgebaut wird (`setProtection`). Anmerkung: Um die Methode `setProtection` für das `source`-Objekt, das den Datentyp `MagicSource` hat, aufrufen zu können, ist eine Typumwandlung (Cast) in den Datentyp `MagicEffectRealization` notwendig. (`((MagicEffectRealization)source)`). Diese Typumwandlung wird im Kontext dieser Aufgabe immer erfolgreich sein.

Sie werden einen Konstruktor für die Klasse benötigen, der dieselben Parameter hat, wie die Basisklasse und einfach alle Parameter an den Konstruktor der Basisklasse weiterreicht. Eventuell benötigen Sie zusätzliche Getter- oder Setter-Methoden.
4. (1 Punkt) Erstellen Sie eine public Methode `boolean checkName(int maxCount)` in der Klasse `Potion`, die genau dann `true` retourniert, wenn im Namen des `this`-Objekts kein Zeichen öfter als `maxCount` mal auftritt. (Groß- und Kleinschreibung werden unterschieden. Um auf den Namen zugreifen zu können, benötigen Sie wahrscheinlich auch eine getter-Methode für den Namen in der Klasse `MagicItem`. Hinweis falls Sie mit Streams arbeiten wollen: Um aus dem Namen einen `Stream<Character>` zu erstellen, können Sie `getName().chars().mapToObj(i->(char)i)` verwenden.)

Kompletieren Sie die in `Main` vordefinierte Methode `selectPotions(List<Potion> list)` so, dass eine Liste aller `Potion`-Objekte in `list`, für die der Aufruf von `Checks.Do_Not_Change.doSelect true` ergibt, retourniert wird. Die relative Reihenfolge der Objekte ist beizubehalten. `doSelect` verlangt einen Parameter, der das in Java vordefinierte funktionale Interface `Function<Integer, Boolean>` mit einer abstrakten Methode `Boolean apply(Integer value)` implementiert. Sie rufen `Checks.Do_Not_Change.doSelect` für `Potion`-Objekte auf, die das Interface `Function<Integer, Boolean>` anbieten und in denen die Methode `apply` so overridden ist, dass `checkName` mit demselben Parameterwert aufgerufen wird. Das kann, da es sich bei `Function` um ein functionales Interface handelt, durch Implementierung des Interfaces in einer geeigneten Klasse, durch Verwenden einer anonymen Klasse oder auch durch einen Lambda-Ausdruck bzw. eine Methoden-Referenz erfolgen.